

Лабораторная работа №1

Студент: Мельник Александр

Группа: 6312-100503D

1. Цель работы

Изучить методы обработки файлов на языке C++, реализовать алгоритм перемножения квадратных матриц, провести исследование зависимости времени выполнения программы от размера задачи, а также реализовать автоматизированную верификацию результатов с использованием стороннего ПО (Python/NumPy).

2. Описание алгоритма

Алгоритм работы программы:

1. Задать массив размеров матриц для исследования: [100, 200, 300, 400, 500]
2. Для каждого размера:
 - Сгенерировать случайные матрицы A и B
 - Считать матрицы из файлов
 - Выполнить перемножение по классическому алгоритму (три вложенных цикла)
 - Замерить время выполнения с использованием <chrono>
 - Сохранить результирующую матрицу в файл
 - Записать метрики (время, объём задачи) в отчёт
3. Сохранить сводные результаты в файл all_results.txt
4. Автоматически верифицировать результат через Python/NumPy

3. Исходный код:

parallel1.c++:

```
#include <iostream>
#include <vector>
#include <fstream>
#include <chrono>
#include <random>

using namespace std;

void generateMatrix(string filename, int n) {
    ofstream file(filename);
    random_device rd;
    mt19937 gen(rd());
    uniform_real_distribution<> dis(1.0, 10.0);

    file << n << endl;
    for (int i = 0; i < n; i++) {
```

```

        for (int j = 0; j < n; j++)
            file << dis(gen) << " ";
        file << endl;
    }
}

vector<vector<double>> readMatrix(string filename, int& n) {
    ifstream file(filename);
    file >> n;
    vector<vector<double>> matrix(n, vector<double>(n));
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            file >> matrix[i][j];
    return matrix;
}

vector<vector<double>> multiply(vector<vector<double>> A, vector<vector<double>>
B) {
    int n = A.size();
    vector<vector<double>> C(n, vector<double>(n, 0));
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            for (int k = 0; k < n; k++)
                C[i][j] += A[i][k] * B[k][j];
    return C;
}

void writeResult(string filename, vector<vector<double>> C) {
    ofstream file(filename);
    int n = C.size();
    file << n << endl;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++)
            file << C[i][j] << " ";
        file << endl;
    }
}

void writeReport(string filename, int n, double time_sec) {
    ofstream file(filename);
    file << "Matrix Size (N): " << n << endl;
    file << "Task Volume (Elements): " << n * n << endl;
    file << "Execution Time (ms): " << time_sec * 1000 << endl;
}

int main() {
    int sizes[] = { 100, 200, 300, 400, 500 };
    int num_sizes = 5;

    ofstream results("all_results.txt");
    results << "N,Time(s),Operations" << endl;

```

```

for (int i = 0; i < num_sizes; i++) {
    int n = sizes[i];
    cout << "Running N=" << n << "... ";

    generateMatrix("matrix_a.txt", n);
    generateMatrix("matrix_b.txt", n);

    int n1, n2;
    auto A = readMatrix("matrix_a.txt", n1);
    auto B = readMatrix("matrix_b.txt", n2);

    auto start = chrono::high_resolution_clock::now();
    auto C = multiply(A, B);
    auto end = chrono::high_resolution_clock::now();

    chrono::duration<double> time = end - start;
    long long ops = 2LL * n * n * n;

    writeResult("result_matrix.txt", C);
    writeReport("report.txt", n, time.count());

    results << n << "," << time.count() << "," << ops << endl;
    cout << time.count() << " s" << endl;
}

results.close();
cout << "\nAll results saved to all_results.txt" << endl;
return 0;
}

```

verify.py:

```

import numpy as np
import matplotlib.pyplot as plt
import os

# Переходим в папку, где лежит скрипт
os.chdir(os.path.dirname(os.path.abspath(__file__)))

# Чтение результатов из all_results.txt
sizes = []
times = []
operations = []

with open("all_results.txt", 'r') as f:
    next(f)
    for line in f:
        parts = line.strip().split(',')
        sizes.append(int(parts[0]))
        times.append(float(parts[1]))

```

```

        operations.append(int(parts[2]))

# Верификация
print("Верификация")
A = np.loadtxt("matrix_a.txt", skiprows=1)
B = np.loadtxt("matrix_b.txt", skiprows=1)
C_result = np.loadtxt("result_matrix.txt", skiprows=1)
C_correct = np.dot(A, B)

if np.allclose(C_result, C_correct, rtol=1e-5):
    print("PASSED")
else:
    print("FAILED")

# График
plt.figure(figsize=(8, 5))
plt.plot(sizes, times, 'bo-', linewidth=2, markersize=8)
plt.xlabel('Размер матрицы N')
plt.ylabel('Время выполнения (с)')
plt.title('Зависимость времени от размера матрицы')
plt.grid(True, alpha=0.3)

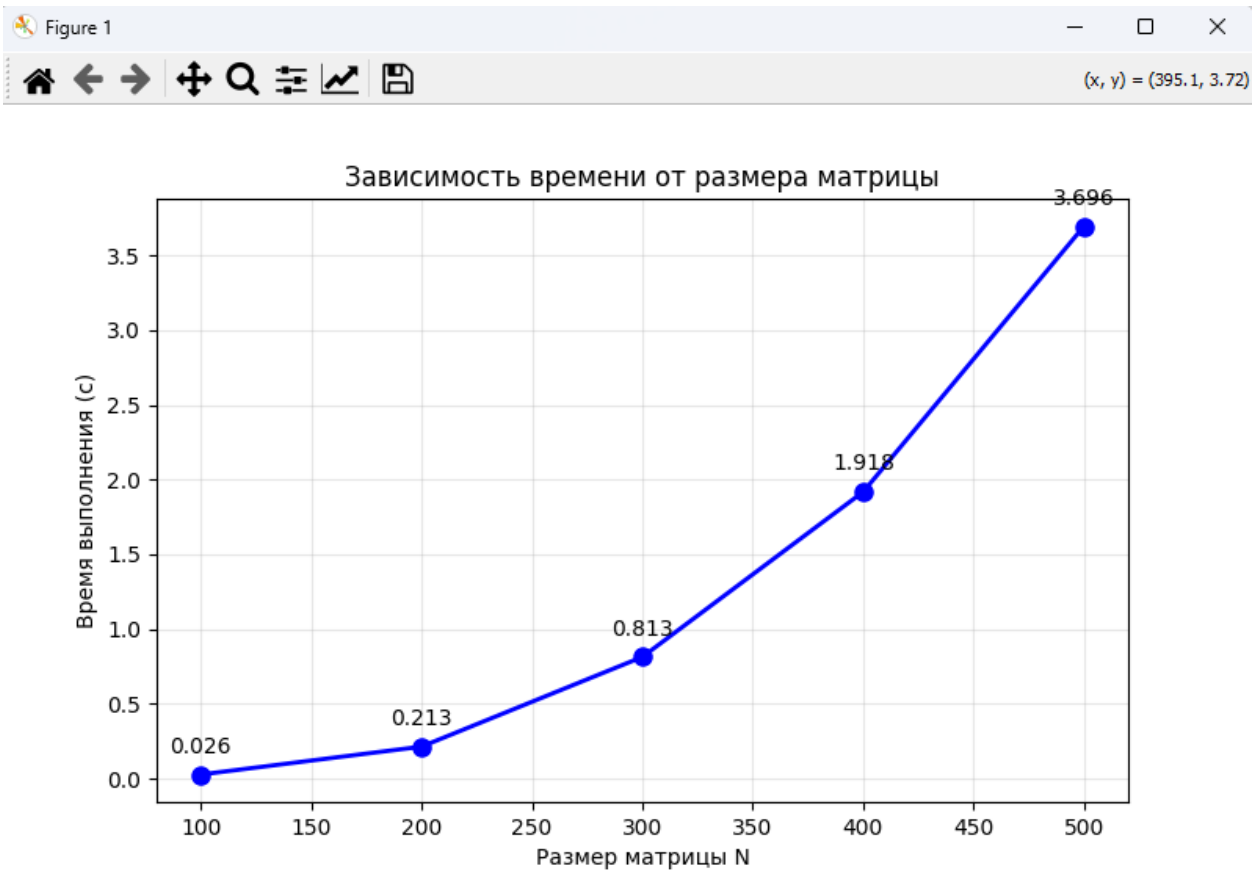
for i, (size, time) in enumerate(zip(sizes, times)):
    plt.annotate(f'{time:.3f}', (size, time), xytext=(0,10),
                 textcoords="offset points", ha='center')

plt.savefig('graph.png', dpi=300, bbox_inches='tight')
plt.show()

# Таблица
print("\nТаблица")
print(f"{'N':<6} {'Операций':<15} {'Время (с)':<10}")
print("-" * 35)
for size, ops, time in zip(sizes, operations, times):
    print(f"{'size':<6} {'ops':<15.2e} {'time:<10.4f}")

```

4. График зависимости



5. Исследование программы

Эксперименты проведены для размеров матриц: 100, 200, 300, 400, 500

Таблица:

N	Операций	Время (с)
100	2.00e+06	0.0263
200	1.60e+07	0.2131
300	5.40e+07	0.8135
400	1.28e+08	1.9183
500	2.50e+08	3.6964

6. Вывод

В ходе выполнения лабораторной работы была разработана программа перемножения квадратных матриц на языке C++ с использованием:

- Файлового ввода/вывода для работы с матрицами
- Библиотеки `<chrono>` для высокоточного замера времени выполнения
- Генерации случайных тестовых данных

Проведено исследование зависимости времени выполнения от размера входных данных. Эксперименты подтвердили, что время выполнения растёт пропорционально N^3 , что полностью соответствует теоретической алгоритмической сложности классического алгоритма умножения матриц.

Также была реализована автоматизированная верификация результатов с использованием языка Python и библиотеки NumPy, что обеспечивает:

- Высокую достоверность вычислений
- Оперативное обнаружение ошибок в реализации
- Возможность регрессионного тестирования при модификации кода